



MyID Verify Technical Guide Document

Document Owner	My Digital ID Sdn. Bhd.
Date	30/9/2026

This document contains confidential and sensitive information. The information contained within should not be reproduced or redistributed without prior written consent from My Digital ID Sdn. Bhd.

TABLE OF CONTENTS

TABLE OF CONTENTS	III
LIST OF TABLES	V
LIST OF FIGURES	VI
SECTION 1: SYSTEM OVERVIEW	1
1 SYSTEM OVERVIEW	2
1.1 MYID VERIFY	2
1.2 USE CASES OF MYID VERIFY	2
1.2.1 MyID Verify – Live	2
1.2.2 MyID Verify – Static	2
1.3 USERS OF THE SYSTEM	2
SECTION 2: MYID VERIFY - STATIC	4
2 MYID VERIFY - STATIC	5
2.1 OVERVIEW	5
2.2 MYID VERIFY – STATIC API	5
2.2.1 Client Credentials	5
2.3 AUTHENTICATION	6
2.3.2 Status Codes	19
2.3.3 Encryption Guide (AES-256-GCM)	20
2.3.4 Response Headers	22
SECTION 3: MYID VERIFY - LIVE	23
3 MYID VERIFY - LIVE	24
3.1 OVERVIEW	24
3.2 CLIENT SETUP REQUIREMENTS	24
3.2.1 Callback URL (`callback_url`)	24
3.2.2 Callback IP Address (`callback_ip`)	24
3.3 CLIENT CREDENTIALS	24
3.3.1 Client ID (`client_id`)	24
3.3.2 Client API Key (`api_key`)	24
3.3.3 Client Secret (`secret`)	24
3.4 AUTHENTICATION	25
3.5 ENDPOINTS	25
3.5.1 Send MyID Verify - Live Request	25

3.5.2 Check MyID Verify - Live Status	28
3.6 STATUS CODES.....	31
SECTION 4: PARTNERS PORTAL	33
4 PARTNERS PORTAL.....	34
4.1 OVERVIEW	34
4.2 SYSTEM FEATURES.....	34
4.2.1 Log In	34
4.2.2 Dashboard	35
4.2.3 Transactions	35
4.2.4 User Billing	38
4.2.5 Change Password	38
APPENDICES	39

LIST OF TABLES

Table 1: Client Credentials	5
Table 2: API Key (Option A)	6
Table 3: Request Form-Data-Fields (Option A)	6
Table 4: Request Body Fields (Option A)	10
Table 5: Request Forms-Data-Fields (Option B)	13
Table 6: Request Body Fields (Option B)	17
Table 7: Static Verification Status Code	19
Table 8: Validation Error Codes	19
Table 9: Encryption Notes	21
Table 10: Response Headers	22
Table 11: Live Status Codes	31
Table 12: Client Callback Status Codes	32
Table 13: HTTP Status Codes	32

LIST OF FIGURES

Figure 1 : Partners Portal Login	34
Figure 2 : Partners Portal Dashboard Screen.....	35
Figure 3 : MyID Verify - Live Transactions Screen	36
Figure 4 : Details of MyID Verify - Live Logs	36
Figure 5 : MyID Verify - Static Transactions Screen.....	37
Figure 6 : Details of MyIDVerify - Static Logs.....	37
Figure 7 : User Billing Screen	38
Figure 8 : Change Password Screen	38

SECTION 1: SYSTEM OVERVIEW

1 SYSTEM OVERVIEW

1.1 MYID VERIFY

MyID Verify is a MyDigital ID component that delivers secure, trusted, and seamless identity verification. It ensures every digital interaction and transaction is backed by a verifiable, authenticated identity.

MyDigital ID offers two verification products: **MyID Verify – Live** and **MyID Verify – Static**.

1. **MyID Verify – Live** provides real-time biometric verification, matching live face biometrics against the JPN national database. This is the highest level of identity assurance, with liveness detection included.
2. **MyID Verify – Static** provides static identity verification using a face image checked against JPN as the single source of truth. This mode is designed for scenarios where KYC and liveness checks have already been completed, assuming a prior liveness check was performed by the Financial Institution (FI).

1.2 USE CASES OF MYID VERIFY

1.2.1 MyID Verify – Live

MyID Verify – Live supports the following use cases:

1. Telco prepaid SIM card activation
2. Telco subscriber whitelisting
3. High-value FI transactions
4. Customer onboarding (regulated sectors)

1.2.2 MyID Verify – Static

MyID Verify – Static supports the following use cases:

1. Post-KYC identity re-confirmation
2. Financial institution document checks
3. Identity binding for existing customers
4. Streamlined onboarding (KYC-complete)

1.3 USERS OF THE SYSTEM

The users of the system involve the following:

1. Integration Team: Technical teams integrating with the platform APIs
2. Billing Portal Users: Operational and finance users managing billing activities

Thus, this manual should be read as follows:

1. Integration Team: **Refer to Section 2 and Section 3**
2. Billing Portal Users: **Refer to Section 4**

SECTION 2: MYID VERIFY - STATIC

2 MYID VERIFY - STATIC

2.1 OVERVIEW

MyID Verify – Static provides static identity verification using a face image checked against JPN as the single source of truth. This mode is designed for scenarios where KYC and liveness checks have already been completed, assuming a prior liveness check was performed by the Financial Institution (FI).

This API does not perform liveness detection, mask detection, or anti-spoofing checks. The system will proceed with face comparison when the face is detected, and sufficient facial features can be extracted from the submitted image.

2.2 MYID VERIFY – STATIC API

This section describes the MyID Verify – Static API endpoints, authentication requirements, request/response formats, encryption specifications, and error handling.

2.2.1 Client Credentials

When your client is created, you will be provided with the following credentials:

Table 1: Client Credentials

Credential	Description
Client ID (client_id)	A UUID that uniquely identifies your application. - Used in the x-client-id header for API calls. - Example: 550e8400-e29b-41d4-a716-446655440000
Client API Key (api_key)	Your authentication key for API access. - Used in the x-api-key header for API calls. - Example: cli_aabc8bf329e110cc77719cb20955bada5b4fe4606b47525b732683457f0c38cec
Client Secret (secret)	Used for AES-256-GCM encryption and hash verification. This is your encryption key — keep it secure and never share it.

2.3 AUTHENTICATION

2.3.1.1 Option A: API Key

All API endpoints require both of the following request headers:

Table 2: API Key (Option A)

Header	Description
x-api-key	The client's API key
x-client-id	The client's unique identifier (UUID)

The system verifies that the API key belongs to the specified client ID.

2.3.1.1.1 Endpoints

2.3.1.1.1.1 Send MyID Verify – Static Request

Performs face verification by comparing a submitted face image against the image from the National Registration IC (MyKad).

```
POST /api/v1/static-verify
Content-Type: multipart/form-data
x-api-key: [client-api-key]
x-client-id: [client-id]
```

Request (Form-Data Fields)

Table 3: Request Form-Data-Fields (Option A)

Field	Type	Required	Description
name	string	Yes	Full name as per IC
ic_no	string	Yes	IC number (numeric only; spaces and special characters are stripped)
enc	string	Yes	SHA-256 hash for integrity verification (see Encryption section)
image	file	Yes	JPEG/JPG or PNG image file (max 7 MB) Min 422 x 305 pixels (Height x Width)
client_ref_id	string	No	Your reference ID for this request
encrypted_name_ic	boolean	No	Default: false. If true, name and ic_no fields are AES-256-GCM encrypted (HEX format)
encrypted_image	boolean	No	Default: false. If true, the image file is AES-256-GCM encrypted binary

Example Requests

Case 1: encrypted_name_ic = false, encrypted_image = false

```
curl -X POST https://<static-verification-domain>/api/v1/static-verify \
-H "x-client-id: your-client-id" \
-H "x-api-key: your-api-key" \
-F "name=ALI BIN ABU" \
-F "ic_no=910203011234" \
-F "enc=sha256_hash_of_name_ic_no_secret" \
-F "client_ref_id=ref-001" \
-F "encrypted_name_ic=false" \
-F "encrypted_image=false" \
-F "image=@/path/to/photo.jpg"
```

Case 2: encrypted_name_ic = true, encrypted_image = true

```
curl -X POST https://<static-verification-domain>/api/v1/static-verify \
-H "x-client-id: your-client-id" \
-H "x-api-key: your-api-key" \
-F "name=46a7b2717cb3a735b0075c48de563f295d687205689172f9a26b671889e1ce6c..." \
-F "ic_no=5b4c749030f3532a194e27408ab795a11b4e7fe5063819cf1b04a4ed16b5696..." \
-F "enc=sha256_hash_of_name_ic_no_secret" \
-F "client_ref_id=ref-001" \
-F "encrypted_name_ic=true" \
-F "encrypted_image=true" \
-F "image=@/path/to/photo.enc"
```

Case 3: encrypted_name_ic = true, encrypted_image = false

```
curl -X POST https://<static-verification-domain>/api/v1/static-verify \
-H "x-client-id: your-client-id" \
-H "x-api-key: your-api-key" \
-F "name=46a7b2717cb3a735b0075c48de563f295d687205689172f9a26b671889e1ce6c..." \
-F "ic_no=5b4c749030f3532a194e27408ab795a11b4e7fe5063819cf1b04a4ed16b5696..." \
-F "enc=sha256_hash_of_name_ic_no_secret" \
-F "client_ref_id=ref-001" \
-F "encrypted_name_ic=true" \
-F "encrypted_image=false" \
-F "image=@/path/to/photo.jpg"
```

Success Responses (HTTP 200)

Face match successful:

```
{
  "success": true,
  "ref_id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "client_ref_id": "ref-001",
  "status": 3100,
  "face_match": true,
  "face_score": 92.45,
  "created_at": "2025-01-19T03:16:57+08:00",
  "response_at": "2025-01-19T03:16:58+08:00"
}
```

Face match failed (image does not match):

```
{
  "success": false,
  "ref_id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "client_ref_id": "ref-001",
  "status": 3101,
  "face_match": false,
  "face_score": 23.18,
  "created_at": "2025-01-19T03:16:57+08:00",
  "response_at": "2025-01-19T03:16:58+08:00"
}
```

Face detection error:

```
{
  "success": false,
  "ref_id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "client_ref_id": "ref-001",
  "status": 3102,
  "face_match": false,
  "face_score": null,
  "created_at": "2025-01-19T03:16:57+08:00",
  "response_at": "2025-01-19T03:16:58+08:00"
}
```

Name and IC number do not match:

```
{
  "success": false,
  "ref_id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "client_ref_id": "ref-001",
  "status": 3103,
  "face_match": false,
  "face_score": null,
  "created_at": "2025-01-19T03:16:57+08:00",
  "response_at": "2025-01-19T03:16:58+08:00"
}
```

Validation Errors (HTTP 400)

```
{
  "status": 3001,
}
```

<pre>"error": "Name is required" }</pre>
<pre>{ "status": 3002, "error": "IC number is required" }</pre>
<pre>{ "status": 3003, "error": "Image file is required" }</pre>
<pre>{ "status": 3004, "error": "Wrong image field name" }</pre>
<pre>{ "status": 3005, "error": "Missing/invalid hash" }</pre>
<pre>{ "status": 3006, "error": "Unable to decrypt name/ic_no" }</pre>
<pre>{ "status": 3007, "error": "Unsupported image format. Only JPEG and PNG allowed" }</pre>
<pre>{ "status": 3008, "error": "Image too large (max 7 MB)" }</pre>
<pre>{ "status": 3009, "error": "Unable to read image file" }</pre>
<pre>{ "status": 3010, "error": "Unable to read image dimensions" }</pre>

Authentication / Authorization Errors (HTTP 401 / 403)

<pre>{ "error": "API key required" }</pre>
<pre>{ "error": "Client ID required in x-client-id header" }</pre>
<pre>{ "error": "Invalid client ID" }</pre>
<pre>{ "error": "Client is inactive" }</pre>
<pre>{</pre>

```
{
  "error": "API key does not match client ID"
}
```

IP Whitelist Error (HTTP 403)

```
{
  "error": "IP not whitelisted",
  "message": "Your IP address is not authorized to access this resource"
}
```

Rate Limit Error (HTTP 429)

```
{
  "success": false,
  "status": 4290,
  "error": "Rate limit exceeded for Static Verification. Maximum 60 requests per minute allowed.",
  "retryAfter": 30,
  "limit": 60,
  "window": 60
}
```

Server Error (HTTP 500)

```
{
  "success": false,
  "status": 3999,
  "error": "Internal server error"
}
```

2.3.1.1.1.2 Get MyID Verify - Static Status

Retrieves the status of a previously submitted MyID Verify – Static request.

```
POST /api/v1/static-verify/status
Content-Type: application/json
x-api-key: [client-api-key]
x-client-id: [client-id]
```

Request Body Fields

Table 4: Request Body Fields (Option A)

Field	Type	Required	Description
ref_id	string	Yes	Reference ID returned from the Send MyID Verify – Static response
ic_no	string	Yes	IC number used in the original request
is_encrypted	boolean	No	Default: false. If true, ic_no is AES-256-GCM encrypted (HEX format)

Example Requests

Case 1: is_encrypted = false

```
curl -X POST https://<static-verification-domain>/api/v1/static-verify/status \
-H "Content-Type: application/json" \
-H "x-client-id: your-client-id" \
-H "x-api-key: your-api-key" \
-d '{
  "ref_id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "ic_no": "910203011234",
  "is_encrypted": false
}'
```

Case 2: is_encrypted = true

```
curl -X POST https://<static-verification-domain>/api/v1/static-verify/status \
-H "Content-Type: application/json" \
-H "x-client-id: your-client-id" \
-H "x-api-key: your-api-key" \
-d '{
  "ref_id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "ic_no": "5b4c749030f3532a194e27408ab795a11b4e7fe5063819cf1b04a4ed16b5696...",
  "is_encrypted": true
}'
```

Success Response (HTTP 200)

```
{
  "ref_id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "client_ref_id": "ref-001",
  "status": 3100,
  "face_score": 92.45,
  "face_match": true,
  "created_at": "2025-01-19T03:16:57+08:00",
  "response_at": "2025-01-19T03:16:58+08:00"
}
```

Validation Errors (HTTP 400)

```
{
  "status": 3021,
  "error": "Ref ID is required"
}

{
  "status": 3022,
  "error": "IC number is required"
}

{
  "status": 3023,
  "error": "Unable to decrypt ic_no"
}
```

Record Not Found (HTTP 404)

```
{
  "status": 3024,
  "error": "Record not found"
}
```

Authentication/Authorization Errors and Rate Limit Errors follow the same format as the Send MyID Verify – Static endpoint.

2.3.1.2 Option B: OAuth2

All API endpoints require a Bearer token obtained via the OAuth2 token endpoint:

2.3.1.2.1 Obtain an Access Token

```
POST /api/v1/oauth/token
Content-Type: application/x-www-form-urlencoded

grant_type=client_credentials&client_id=<your-client-id>&client_secret=<your-client-secret>
```

2.3.1.2.1.1 Success Response (200)

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIs... ",
  "token_type": "Bearer",
  "expires_in": 900
}
```

2.3.1.2.1.2 Error Responses (400)

```
{
  "error": "invalid_client",
  "error_description": "Invalid client credentials"
}
{
  "error": "invalid_request",
  "error_description": "client_id is required"
}
{
  "error": "unsupported_grant_type",
  "error_description": "Only client_credentials grant type is supported"
}
```

2.3.1.2.2 Use the Token

Include the token in the `Authorization` header for all API calls:

```
Authorization: Bearer <access_token>
```

Important: The token expires after the `expires\_in` period (default: 15 minutes). Obtain a new token before it expires. The old token becomes invalid immediately upon expiry.

2.3.1.2.3 Endpoints

2.3.1.2.3.1 Send MyID Verify - Static

Performs face verification by comparing a submitted face image against the image from the National Registration IC.

```
POST /api/v1/static-verify
Content-Type: multipart/form-data
Authorization: Bearer <access_token>
```

Request (Form-Data Fields)

Table 5: Request Forms-Data-Fields (Option B)

Field	Type	Required	Description
`name`	string	Yes	Full name as per IC
`ic_no`	string	Yes	IC number (numeric only, spaces/special chars will be stripped)
`enc`	string	Yes	SHA256 hash for integrity verification (see Encryption section)
`image`	file	Yes	JPEG or PNG image file (max 7MB)
`client_ref_id`	string	No	Your reference ID for this request
`encrypted_name_ic`	boolean	No	Default: `false`. If `true`, `name` & `ic_no` fields are AES-256-GCM encrypted (HEX format) (see Encryption section)
`encrypted_image`	boolean	No	Default: `false`. If `true`, `image` file is AES-256-GCM encrypted binary (see Encryption section)

Example Requests – Non-encrypted

Step 1 : Get Token

```
TOKEN=$(curl -s -X POST https://<domain>/api/v1/oauth/token \
-d "grant_type=client_credentials" \
-d "client_id=your-client-id" \
-d "client_secret=your-client-secret" \
| jq -r '.access_token')
```

Step 2 : Use Token

```
curl -X POST https://<domain>/api/v1/static-verify \  
-H "Authorization: Bearer $TOKEN" \  
-F "name=ALI BIN ABU" \  
-F "ic_no=910203011234" \  
-F "enc=sha256_hash_of_name_ic_no_secret" \  
-F "client_ref_id=ref-001" \  
-F "encrypted_name_ic=false" \  
-F "encrypted_image=false" \  
-F "image=@/path/to/photo.jpg"
```

Example Requests – Non-encrypted

```
TOKEN=$(curl -s -X POST https://<domain>/api/v1/oauth/token \  
-d "grant_type=client_credentials" \  
-d "client_id=your-client-id" \  
-d "client_secret=your-client-secret" \  
| jq -r '.access_token')  
  
curl -X POST https://<domain>/api/v1/static-verify \  
-H "Authorization: Bearer $TOKEN" \  
-F "name=46a7b2717cb3a735b0075c48de563f295d687205689172f9a26b671889e1ce6c3357d7859c4cd2" \  
-F "ic_no=5b4c749030f3532a194e27408ab795a11b4e7fe5063819cf1b04a4ed16b5696780bcefcaa46509d2" \  
-F "enc=sha256_hash_of_name_ic_no_secret" \  
-F "client_ref_id=ref-001" \  
-F "encrypted_name_ic=true" \  
-F "encrypted_image=true" \  
-F "image=@/path/to/photo.enc"
```

Success Responses (200)

Face match successful:

```
{
  "success": true,
  "ref_id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "client_ref_id": "ref-001",
  "status": 3100,
  "face_match": true,
  "face_score": 92.45,
  "created_at": "2025-01-19T03:16:57+08:00",
  "response_at": "2025-01-19T03:16:58+08:00"
}
```

Face match failed (image does not match):

```
{
  "success": false,
  "ref_id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "client_ref_id": "ref-001",
  "status": 3101,
  "face_match": false,
  "face_score": 23.18,
  "created_at": "2025-01-19T03:16:57+08:00",
  "response_at": "2025-01-19T03:16:58+08:00"
}
```

Face detection error:

```
{
  "success": false,
  "ref_id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "client_ref_id": "ref-001",
  "status": 3102,
  "face_match": false,
  "face_score": null,
  "created_at": "2025-01-19T03:16:57+08:00",
  "response_at": "2025-01-19T03:16:58+08:00"
}
```

Name and IC number do not match:

```
{
  "success": false,
  "ref_id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "client_ref_id": "ref-001",
  "status": 3103,
  "face_match": false,
  "face_score": null,
  "created_at": "2025-01-19T03:16:57+08:00",
  "response_at": "2025-01-19T03:16:58+08:00"
}
```

Validation Errors (HTTP 400)

```
{
  "status": 3001,
}
```

<pre>"error": "Name is required" }</pre>
<pre>{ "status": 3002, "error": "IC number is required" }</pre>
<pre>{ "status": 3003, "error": "Image file is required" }</pre>
<pre>{ "status": 3004, "error": "Wrong image field name" }</pre>
<pre>{ "status": 3005, "error": "Missing/invalid hash" }</pre>
<pre>{ "status": 3006, "error": "Unable to decrypt name/ic_no" }</pre>
<pre>{ "status": 3007, "error": "Unsupported image format. Only JPEG and PNG allowed" }</pre>
<pre>{ "status": 3008, "error": "Image too large (max 7 MB)" }</pre>
<pre>{ "status": 3009, "error": "Unable to read image file" }</pre>
<pre>{ "status": 3010, "error": "Unable to read image dimensions" }</pre>

Authentication Errors (HTTP 401)

<pre>{ "error": "Invalid or expired token" }</pre>
<pre>{ "error": "JWT_SECRET not configured" }</pre>

Authorization Errors (HTTP 403)

<pre>{ "error": "IP not whitelisted", "message": "Your IP address is not authorized to access this resource" }</pre>
--

Rate Limit Error (HTTP 429)

```
{
  "success": false,
  "status": 4290,
  "error": "Rate limit exceeded for Static Verification. Maximum 60 requests per minute
allowed.",
  "retryAfter": 30,
  "limit": 60,
  "window": 60
}
```

Server Error (HTTP 500)

```
{
  "success": false,
  "status": 3999,
  "error": "Internal server error"
}
```

2.3.1.2.3.2 Get MyID Verify - Static Status

Retrieves the status of a previously submitted MyID Verify - Static request.

```
POST /api/v1/static-verify/status
Content-Type: application/json
Authorization: Bearer <access_token>
```

Request Body Fields**Table 6: Request Body Fields (Option B)**

Field	Type	Required	Description
ref_id	string	Yes	Reference ID returned from the Send MyID Verify - Static response
ic_no	string	Yes	IC number used in the original request
is_encrypted	boolean	No	Default: false. If true, ic_no is AES-256-GCM encrypted (HEX format)

Example Requests

```
TOKEN=$(curl -s -X POST https://<domain>/api/v1/oauth/token \
-d "grant_type=client_credentials" \
-d "client_id=your-client-id" \
-d "client_secret=your-client-secret" \
| jq -r '.access_token')
```

```
curl -X POST https://<domain>/api/v1/static-verify/status \
-H "Content-Type: application/json" \
-H "Authorization: Bearer $TOKEN" \
-d '{
  "ref_id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "ic_no": "910203011234",
  "is_encrypted": false
}'
```

Success Response (HTTP 200)

```
{
  "ref_id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "client_ref_id": "ref-001",
  "status": 3100,
  "face_score": 92.45,
  "face_match": true,
  "created_at": "2025-01-19T03:16:57+08:00",
  "response_at": "2025-01-19T03:16:58+08:00"
}
```

Validation Errors (HTTP 400)

```
{
  "status": 3021,
  "error": "Ref ID is required"
}
```

```
{
  "status": 3022,
  "error": "IC number is required"
}
```

```
{
  "status": 3023,
  "error": "Unable to decrypt ic_no"
}
```

Record Not Found (HTTP 404)

```
{
  "status": 3024,
  "error": "Record not found"
}
```

Authentication and Rate Limit Errors follow the same format as the Send Static Verification endpoint.

Server Error (HTTP 500)

```
{
  "success": false,
  "status": 3999,
  "error": "Internal server error"
}
```

2.3.2 Status Codes

Static Verification Status Codes

Table 7: Static Verification Status Code

Code	Description
3100	Face match successful
3101	Face match failed
3102	Face detection error
3103	Verification hash failed (Name/IC mismatch)
3199	Unknown error
3999	Internal server error

Validation Error Codes

Table 8: Validation Error Codes

Code	Description
3001	Name is required
3002	IC number is required
3003	Image file is required
3004	Wrong image field name
3005	Missing/invalid hash
3006	Unable to decrypt name/ic_no
3007	Unsupported image format
3008	Image too large (max 7 MB)
3009	Unable to read image file
3010	Unable to read image dimensions
3021	Ref ID is required
3022	IC number is required (status query)
3023	Unable to decrypt ic_no
3024	Record not found
4290	Rate limit exceeded

2.3.3 Encryption Guide (AES-256-GCM)

This section describes the AES-256-GCM encryption scheme used to protect sensitive request fields. The encryption key is derived from your client secret.

2.3.3.1 Name and IC Number Encryption

When `encrypted_name_ic` is set to true, the `name` and `ic_no` fields must each be individually encrypted using AES-256-GCM before submission.

Encryption format: IV (12 bytes) + AuthTag (16 bytes) + Ciphertext — output as HEX string

```
const crypto = require("crypto");

// Derive 32-byte AES key from client secret using SHA-256
const getAesKey = (clientSecret) => {
  return crypto.createHash("sha256").update(clientSecret).digest();
};

// Encrypt plaintext and return HEX string
const AESEncrypt = (text, clientSecret) => {
  const key = getAesKey(clientSecret);
  const iv = crypto.randomBytes(12);
  const cipher = crypto.createCipheriv("aes-256-gcm", key, iv);
  let encrypted = cipher.update(text, "utf8");
  encrypted = Buffer.concat([encrypted, cipher.final()]);
  const tag = cipher.getAuthTag();
  return Buffer.concat([iv, tag, encrypted]).toString("hex");
};
```

2.3.3.2 Image File Encryption

When `encrypted_image` is set to true, the image file must be encrypted as a binary blob. The encrypted file is uploaded as the multipart form-data image field.

Encryption format: IV (12 bytes) + AuthTag (16 bytes) + Ciphertext — output as raw binary file

```
const crypto = require("crypto");
const fs = require("fs");

const getAesKey = (clientSecret) => {
  return crypto.createHash("sha256").update(clientSecret).digest();
};

// Encrypt image file – output is raw binary
const encryptImageFile = (inputPath, outputPath, clientSecret) => {
  const key = getAesKey(clientSecret);
  const iv = crypto.randomBytes(12);
```

```

const cipher = crypto.createCipheriv("aes-256-gcm", key, iv);
const input = fs.readFileSync(inputPath);
let encrypted = cipher.update(input);
encrypted = Buffer.concat([encrypted, cipher.final()]);
const tag = cipher.getAuthTag();
fs.writeFileSync(outputPath, Buffer.concat([iv, tag, encrypted]));
};

// Decrypt image file – for verification purposes
const decryptImageFile = (inputPath, outputPath, clientSecret) => {
  const key = getAesKey(clientSecret);
  const data = fs.readFileSync(inputPath);
  const iv = data.subarray(0, 12);
  const tag = data.subarray(12, 28);
  const enc = data.subarray(28);
  const decipher = crypto.createDecipheriv("aes-256-gcm", key, iv);
  decipher.setAuthTag(tag);
  const decrypted = Buffer.concat([decipher.update(enc), decipher.final()]);
  fs.writeFileSync(outputPath, decrypted);
};

```

2.3.3.3 Hash Calculation (enc Field)

The enc field provides integrity verification for the name and IC number fields. It must be computed as follows:

```

const enc = crypto.createHash("sha256")
  .update(`name${name}ic_no${ic_no}${client_secret}`)
  .digest("hex");

```

2.3.3.4 Important Encryption Notes

Table 9: Encryption Notes

Note	Detail
Encryption Key	The client_secret is used to derive the AES-256 key via SHA-256
Separate Encryption	Encrypt name and ic_no independently — do not encrypt them together
Output Format (Name/IC)	Encrypted name and ic_no output must be a HEX string
Output Format (Image)	Encrypted image file output must be raw binary
Flag Requirement	When sending encrypted data, the corresponding flags (encrypted_name_ic, encrypted_image) must be set to true

2.3.4 Response Headers

Rate limiting information is included in the following response headers:

Table 10: Response Headers

Header	Description
X-RateLimit-Limit	Maximum number of requests allowed per time window
X-RateLimit-Remaining	Number of requests remaining in the current window
X-RateLimit-Window	Time window duration in seconds
X-RateLimit-Reset	Seconds until the rate limit resets (only present when rate limit is exceeded)

SECTION 3: MYID VERIFY - LIVE

3 MYID VERIFY - LIVE

3.1 OVERVIEW

MyID Verify – Live provides real-time biometric verification, matching live face biometrics against the JPN national database. This is the highest level of identity assurance, with liveness detection included.

3.2 CLIENT SETUP REQUIREMENTS

Before your client application can be created, you need to provide the following information to the MyDigital ID admin:

3.2.1 Callback URL (``callback_url``)

The endpoint where our system will send notifications.

- Example: `'https://your-domain.com/webhook/myid-notifications'`
- Must be a publicly accessible HTTPS URL
- Should be able to handle POST requests with JSON payload
- This is where e-verification status updates will be sent

3.2.2 Callback IP Address (``callback_ip``)

Your server's IP address for security

- Example: `'203.0.113.10'`
- Used for IP whitelisting and security verification
- Can be IPv4 or IPv6 address
- Ensure this IP is static or update us when it changes

3.3 CLIENT CREDENTIALS

When your client application is created by the admin, you will be provided with three important credentials:

3.3.1 Client ID (``client_id``)

A UUID that uniquely identifies your application.

- Example: `'550e8400-e29b-41d4-a716-446655440000'`
- Used in `'x-client-id'` header for API calls

3.3.2 Client API Key (``api_key``)

Your authentication key for API access.

- Example: `'cli_aabc8bf329e110cc77719cb20955bada5b4fe4606b47525b732683457f0c38cec'`
- Used in `'x-api-key'` header for API calls

3.3.3 Client Secret (``secret``)

A 12-character secret for callback verification.

- Example: `12-character-auto-generated-secure-secret...`
- Used to verify the authenticity of callbacks from our system
- Never include this in API requests – it is only for callback verification

3.4 AUTHENTICATION

Client authentication requires BOTH headers:

- `x-api-key` : The client's API key
- `x-client-id` : The client's UUID

The system will verify that the API key belongs to the specified client ID.

3.5 ENDPOINTS

3.5.1 Send MyID Verify - Live Request

```
POST /api/v1/notify/everification
Content-Type: application/json
x-api-key: [client-api-key]
x-client-id: [client-uuid]
```

Body:

If `is\_encrypted: false`

```
{
  "name": "ALI BIN ABU",
  "ic_no": "910203011234"
  "enc": "sha256_hash_of_name_ic_no_secret",
  "client_ref_id": "client_provided_ref_id", // optional
  "is_encrypted": false, // optional, default: false. If `true`, name & ic no must be in AES-
256-GCM HEX format
  "svc_type": "00000000", // optional, Please refer to your account manager on the service
type
}
```

If `is\_encrypted: true`

```
{
  "name": "46a7b2717cb3a735b0075c48de563f295d687205689172f9a26b671889e1ce6c3357d7859c4cd2",
  "ic_no": "5b4c749030f3532a194e27408ab795a11b4e7fe5063819cf1b04a4ed16b5696780bcefc9aa46509d2"
  "enc": "sha256_hash_of_name_ic_no_secret",
  "client_ref_id": "client_provided_ref_id", // optional
  "is_encrypted": true, // optional, default: false. If `true`, name & ic no must be in AES-
256-GCM HEX format
  "svc_type": "00000000", // optional, Please refer to your account manager on the service
type
}
```

Success Response (200):

```
{
  "success": true,
  "ref_id": "uuid", // autogenerated
  "client_ref_id": "client_provided_ref_id",
  "status": 1000, // 1000 = success, 1001 = failed
  "created_at": "2025-01-19T03:16:57+08:00",
  "expired_at": "2025-01-19T03:19:57+08:00", // 3 minutes from created_at
  "device_platform": "android"
}
```

Error Responses:

```
// Authentication Errors (401)
{
  "error": "API key required"
}
{
  "error": "Client ID required in x-client-id header"
}

// Authorization Errors (403)
{
  "error": "Invalid client ID"
}
{
  "error": "Client is inactive"
}
{
  "error": "API key does not match client ID"
}

// IP Whitelist Error (403)
{
  "error": "IP not whitelisted",
  "message": "Your IP address is not authorized to access this resource"
}

// Rate Limit Error (429)
{
  "success": false,
  "status": 4290,
  "error": "Rate limit exceeded for eVerification. Maximum 60 requests per minute allowed.",
  "retryAfter": 30, // seconds until rate limit resets
  "limit": 60, // maximum requests allowed in window
  "window": 60 // time window in seconds
}

// Validation Errors (400)
{
  "status": 1002,
  "error": "Name is required"
}
{
  "status": 1003,
  "error": "IC number is required"
}
{
  "status": 1006,
```

```
"error": "Missing/invalid hash"
}
{
  "status": 1007,
  "error": "Unable to decrypt name/ic no"
}

// Device Not Found Error (404)
{
  "status": 1005,
  "error": "Device not found for this IC number"
}

// Server Error (500)
{
  "error": "Internal server error"
}
```

Response Headers

Rate limiting information is included in response headers:

1. `X-RateLimit-Limit`: Maximum requests allowed per window
2. `X-RateLimit-Remaining`: Number of requests remaining in current window
3. `X-RateLimit-Window`: Time window in seconds
4. `X-RateLimit-Reset`: Seconds until rate limit resets (only when rate limit is exceeded)

Security Note:

1. The `enc` field contains a SHA256 hash of the concatenated string: `name{name}ic\_no{ic\_no}{client\_secret}`. This allows you to verify the authenticity of the callback request.
2. - If `is\_encrypted = true`, use `client\_secret` for the secret key in AES-256-GCM generation.

Important Note on Encryption:

1. Hash Calculation: Always calculate the `enc` hash using the **original (decrypted)** values:

```
// CORRECT ✓
const enc = SHA256(`name${original_name}ic_no${original_ic_no}${client_secret}`);

// WRONG ✗
const enc = SHA256(`name${encrypted_name}ic_no${encrypted_ic_no}${client_secret}`);
```

2. Encryption Key: The `client\_secret` is used as the encryption key (after SHA256 hashing to derive the AES-256 key)
3. Separate Encryption: Encrypt `name` and `ic\_no` separately - do not encrypt them together
4. Output Format: The encrypted output must be in HEX string format
5. Flag Requirement: When sending encrypted data, you **MUST** set `is\_encrypted: true`

Verification Hash Example:

```
// To verify the callback authenticity
const expectedString = `name${payload.name}ic_no${payload.ic_no}${your_client_secret}`;
const expectedHash = crypto.createHash('sha256').update(expectedString).digest('hex');
if (expectedHash === payload.enc) {
  // Callback is authentic
}
```

Verification Name & IC No. Encryption (AES-256-GCM) Example:

```
// ◊ Derive 32-byte AES key using SHA256
const getAesKey = (clientSecret) => {
  return createHash("sha256").update(clientSecret).digest(); // Buffer (32 bytes)
}

// ◊ Encrypt and output hex
const AESEncrypt = (text, clientSecret) => {
  const key = getAesKey(clientSecret);
  const iv = randomBytes(12);
  const cipher = createCipheriv("aes-256-gcm", key, iv);

  let encrypted = cipher.update(text, "utf8");
  encrypted = Buffer.concat([encrypted, cipher.final()]);

  const tag = cipher.getAuthTag();

  // Combine IV + encrypted data + tag → hex string
  return Buffer.concat([iv, tag, encrypted]).toString("hex");
}
```

3.5.2 Check MyID Verify - Live Status

```
POST /api/v1/everification/status
Content-Type: application/json
x-api-key: [client-api-key]
x-client-id: [client-uuid]
```

Body:

If `is\_encrypted: false`

```
{
  "ic_no": "910203011234",
  "ref_id": "3bc4ad50-c75e-4fdd-9cab-9e2d99d1982f", // Ref ID generated from everification request response
  "is_encrypted": false, // optional, default: false. If `true`, ic no must be in AES-256-GCM HEX format
}
```

If ``is_encrypted: true``

```
{
  "ic_no": "5b4c749030f3532a194e27408ab795a11b4e7fe5063819cf1b04a4ed16b5696780bcefcaa46509d2",
  "ref_id": "3bc4ad50-c75e-4fdd-9cab-9e2d99d1982f", // Ref ID generated from everification
  request response
  "is_encrypted": true, // optional, default: false. If `true`, ic no must be in AES-256-GCM
  HEX format
}
```

Success Response (200):

```
{
  "ref_id": "3bc4ad50-c75e-4fdd-9cab-9e2d99d1982f",
  "client_ref_id": "12345",
  "action": "everification",
  "hash": "dfe067e4de51dbacae2e31ed68443a1598f05cb04f130409b01d2b287ecbe1f", // hash of name &
  ic no
  "score": 0, // face matching score
  "status": 2004, // will return code 1000, 200x
  "created_at": "2025-11-05T15:13:18+08:00",
  "expired_at": "2025-11-05T15:16:18+08:00",
  "response_at": "2025-11-05T15:13:53+08:00" // Time of response from MyDigital ID app, `null`
  if none
}
```

Error Responses:

```
// Authentication Errors (401)
{
  "error": "API key required"
}
{
  "error": "Client ID required in x-client-id header"
}

// Authorization Errors (403)
{
  "error": "Invalid client ID"
}
{
  "error": "Client is inactive"
}
{
  "error": "API key does not match client ID"
}

// IP Whitelist Error (403)
{
  "error": "IP not whitelisted",
  "message": "Your IP address is not authorized to access this resource"
}

// Rate Limit Error (429)
{
  "success": false,
  "status": 4290,
  "error": "Rate limit exceeded for eVerification. Maximum 120 requests per minute allowed.",
  "retryAfter": 30, // seconds until rate limit resets
}
```

```
"limit": 120, // maximum requests allowed in window
>window": 60 // time window in seconds
}

// Validation Errors (400)
{
  "status": 1003,
  "error": "IC number is required"
}
{
  "status": 1007,
  "error": "Unable to decrypt name/ic no"
}
{
  "status": 1008,
  "error": "Ref ID is required"
}

// Record Not Found Error (404)
{
  "status": 1009,
  "error": "Record not found"
}

// Server Error (500)
{
  "error": "Internal server error"
}
```

Important Note on Encryption:

Use the same AES-256-GCM encryption for ic no. as in send everification request api

Client Callback (Outgoing from Our System)

When an everification status is updated from our eVerification server, our system will automatically send a callback request to the client's registered `callback` URL:

```
POST {client.callback}
Content-Type: application/json

Body:
{
  "ref_id": "uuid", // Generated from Send EVerification/Login request
  "client_ref_id": "client_provided_ref_id",
  "hash": "hash_of_name_ic",
  "score": 86.19, // face matching score
  "status": 2000, // updated status code (2000 = success, !2000 = failed/error)
  "timestamp": "2025-01-19T10:44:43+08:00",
  "enc": "sha256_hash_of_hash_ref_id_score_status_secret"
}
```

Security Note: The `enc` field contains a SHA256 hash of the concatenated string: `hash{hash}ref\_id{ref\_id}score{score}status{status}{client\_secret}`. This allows you to verify the authenticity of the callback request.

Verification Example

```
// To verify the callback authenticity
const expectedString =
`hash${payload.hash}ref_id${payload.ref_id}score${payload.score}status${payload.status}${your_client_secret}`;
const expectedHash = crypto.createHash('sha256').update(expectedString).digest('hex');
if (expectedHash === payload.enc) {
  // Callback is authentic
}
```

3.6 STATUS CODES

MyID Verify - Live Status Codes

Table 11: Live Status Codes

Code	Description
1000	Success
1001	Failed
1002	Name is required
1003	IC No. is required
1004	QR data is required
1005	Device not found for this IC number
1006	Missing/invalid hash
1007	Unable to decrypt name/ic no
1008	Ref ID is required
1009	Record not found for this ref id & ic no
4290	Rate limit exceeded

Client Callback Status Codes

Table 12: Client Callback Status Codes

Code	Description
2000	Success
2001	EVerification Failed
2002	Name & IC no. not match
2003	Request expired
2004	Cert expired
2005	User cancelled
2200`	SDK Error

HTTP Status Codes

Table 13: HTTP Status Codes

Code	Description
200	Success
400	Bad Request (validation error)
401	Unauthorized (missing or invalid API key/client ID)
403	Forbidden (inactive client, API key mismatch, or IP not whitelisted)
404	Not Found (device not registered)
429	Too Many Requests (rate limit exceeded)
500	Internal Server Error

SECTION 4: PARTNERS PORTAL

4 PARTNERS PORTAL

4.1 OVERVIEW

Partners Portal is a dashboard or interface offered to MyDigital ID partners to view real-time or periodic status of MyID Verify checks. It allows the partners to review MyID Verify logs to satisfy compliance or fulfill internal risk-management requirements

Partners Portal provides following features:

1. Dashboard
2. Transactions
3. User Billing
4. Change Password
5. System Settings

4.2 SYSTEM FEATURES

4.2.1 Log In

Partners need to login to the portal by using the username and password provided or Login using MyDigital ID (if user has registered with partner portal).

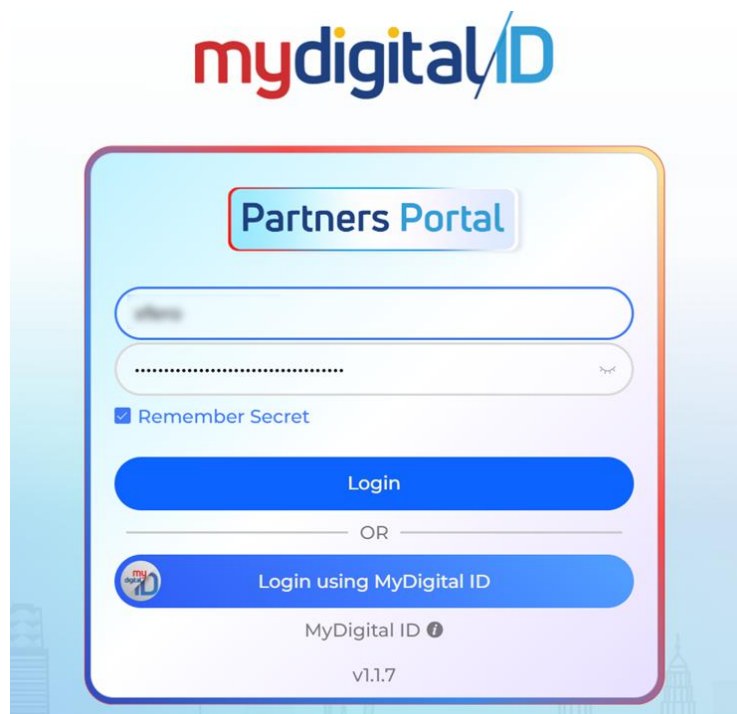


Figure 1 : Partners Portal Login

4.2.2 Dashboard

The Dashboard module displays the summary of MyID Verify activities in charts, grouped by status and face score.

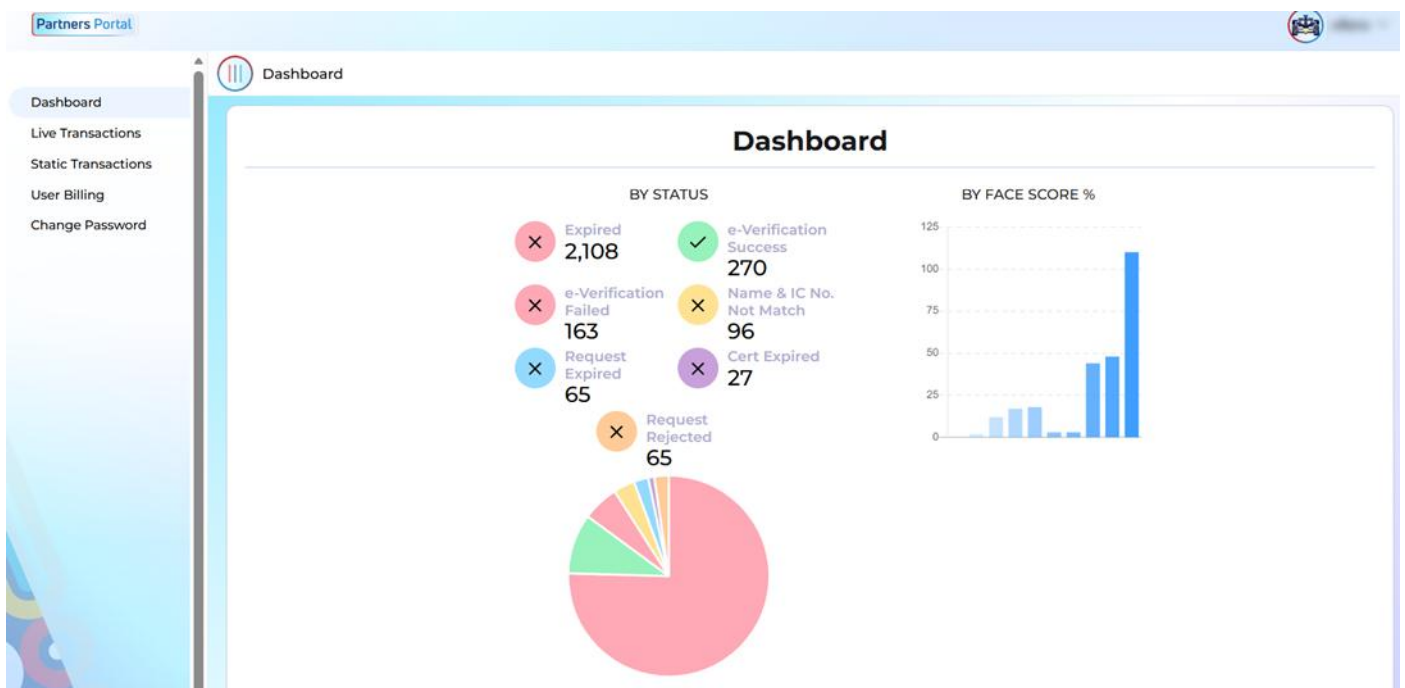


Figure 2 : Partners Portal Dashboard Screen

4.2.3 Transactions

The Transactions module provides the list of MyID Verify transaction logs. Partners can filter by start and end dates as well as by transaction status. Partners also can search by IC number, export the logs into CSV format and view the details of each log.

4.2.3.1 Live Transactions

Live transactions is a list of transactions generated when a user using MyIDVerify - Live.

No.	User	Status	Face Score	Ref ID	Client Ref ID	Action	Created At
1		Expired 1000	-	9dcd282f-b6e9...	-	everification	25/05/2026 11:07:19
2		Expired 1000	-	e10ce357-3ffa-4...	-	everification	11/05/2026 16:23:01
3		Expired 1000	-	e85a7b4c-0775...	-	everification	11/05/2026 10:27:13
4		Expired 1000	-	0d12c17b-a538-...	-	everification	11/05/2026 10:25:43
5		Expired 1000	-	45cbf112-cdf2-4...	-	everification	11/05/2026 10:24:42
6		Expired 1000	-	ed8a1bc3-0037...	-	everification	11/05/2026 10:24:11
7		Expired 1000	-	8470ff1c-6011-4...	-	everification	23/04/2026 16:40:57

Figure 3 : MyID Verify - Live Transactions Screen

View Details

User: [Redacted]

Ref ID: [Redacted]

Client Ref ID: [Redacted]

Client ID: [Redacted]

Action: verification

Created At: 11/05/2026 10:27:13

Expired At: 11/05/2026 10:30:13

Response At: Invalid date

Status: Expired

Status Code: 1000

Close

Figure 4 : Details of MyID Verify - Live Logs

4.2.3.2 Static Transactions

Static transactions is a list of transactions generated when user perform MyID Verify - Static.

No.	User	Status	Face Score	Face Match	Ref ID	Client Ref ID	Created At
1	[User Icon]	Face Match Failed 3101	48.59	No	7cf59502-b0bd...	-	30/06/2026 16:13:41
2	[User Icon]	Face Match Failed 3101	48.59	No	d310c55b-14d0...	-	29/06/2026 18:36:56
3	[User Icon]	Face Match Failed 3101	48.59	No	9fe4a275-f6e9...	-	29/06/2026 18:33:52
4	[User Icon]	Name & IC No. Not Match	-	No	f6fd4290-3274...	-	29/06/2026 18:33:25

Figure 5 : MyID Verify - Static Transactions Screen

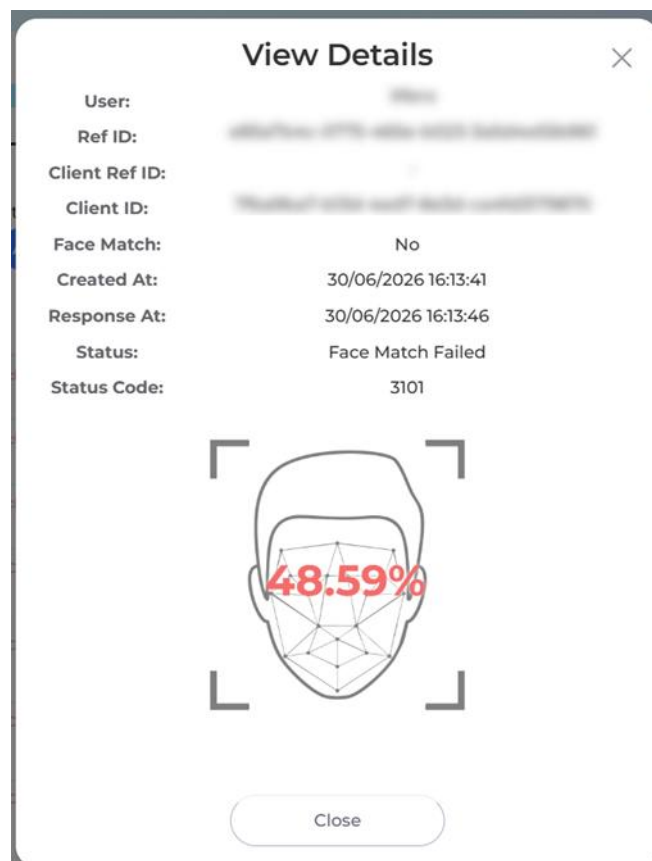
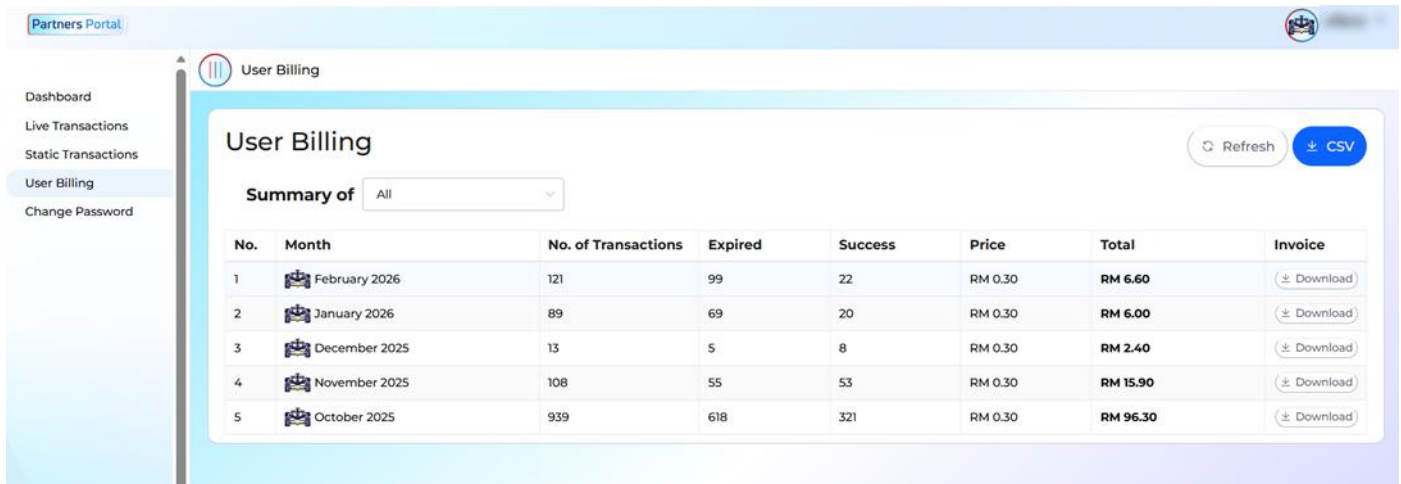


Figure 6 : Details of MyIDVerify - Static Logs

4.2.4 User Billing

User Billing module allows the Partners to view the charging details based on the MyID Verify transactions according to billing date.



No.	Month	No. of Transactions	Expired	Success	Price	Total	Invoice
1	February 2026	121	99	22	RM 0.30	RM 6.60	Download
2	January 2026	89	69	20	RM 0.30	RM 6.00	Download
3	December 2025	13	5	8	RM 0.30	RM 2.40	Download
4	November 2025	108	55	53	RM 0.30	RM 15.90	Download
5	October 2025	939	618	321	RM 0.30	RM 96.30	Download

Figure 7 : User Billing Screen

4.2.5 Change Password

Change Password module allows Partners to change the password to access the Partners portal.

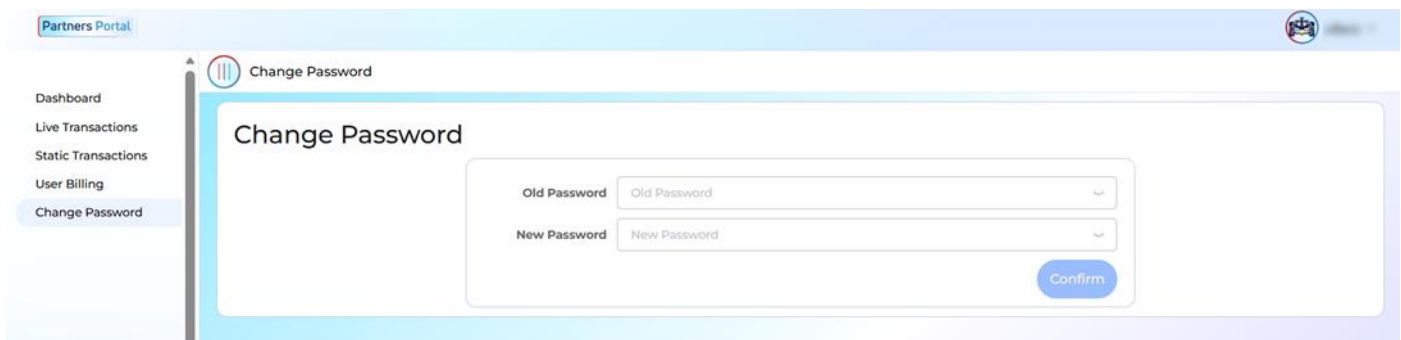


Figure 8 : Change Password Screen

APPENDICES

svc_type Parameter — Quick Reference

For clients integrating with the MyDigital ID Live e-Verification / Login API

WHAT IT IS

An optional field, `svc_type`, included in the request body when calling the Send e-Verification or Send Login endpoints. It identifies the industry, service type, and client submitting the request.

FORMAT — 9 DIGITS, 3 SEGMENTS

000 + 000 + 000
INDUSTRY TYPE OF SERVICE CLIENT ID

The three 3-digit codes are joined with no separators to form the final value, e.g. `001 + 002 + 045 = 001002045`.

INDUSTRY CODES

CODE	INDUSTRY
001	Telecommunication
002	Financial institution

TYPE OF SERVICE CODES

CODE	TYPE OF SERVICE
001	User whitelisting
002	User registration

Client ID codes are assigned individually — contact your account manager for your value.

EXAMPLE

```
{
  "name": "ALI BIN ABU",
  "ic_no": "91[REDACTED]4",
  "enc": "sha256_hash_of_name_ic_no_secret",
  "svc_type": "001002045" // industry + type_of_service + client_id
}
```

